

Math for PS

2026-1학기 부경대학교 알고리즘 스터디

목차

- 기초 정수론 지식 정리

- 용어 정리: 약수, 배수, 인수, 소수, 합성수 등..

- 나머지 정리

- 공약수와 최대공약수

- 서로소와 소인수분해

- 모듈러 연산

- Square Root 테크닉

- 에라토스테네스의 체

- 유클리드 호제법

- 이항 계수 (with DP)

- 빠른 거듭제곱 (with 분할정복)

정수론 기본 용어 정리

- 두 정수 a 와 d 에 대해 " a 를 d 로 나눌 수 있다"는 $a = kd$ 를 만족시키는 어떤 정수 k 가 존재한다는 뜻이다.
 - 이러한 관계를 $d \mid a$ 로 나타낸다.
 - 이 때 $d \geq 0$ 인 d 를 a 의 약수(divisor)라고 칭한다.
 - 이 때 $a > 0$ 인 a 를 d 의 배수(multiple)라고 칭한다.
- 모든 양의 정수는 1과 자기 자신으로 나눌 수 있다.
 - 이러한 1과 자기 자신을 명백한 약수(trivial divisor)라고 한다.
 - 명백한 약수가 아닌 약수를 인수(factor)라고 한다.
- 위 내용을 토대로 다음을 정의하자.
 - $a > 1$ 인 정수에 대해 명백한 약수만 존재하는 수를 소수(prime number)라고 한다.
 - $a > 1$ 인 정수에 대해 소수가 아닌 수를 합성수(composite number)라고 한다.

나머지 정리

- 정수 a 와 양의 정수 n 에 대해 $0 \leq r < n$ 이고, $a = qn + r$ 을 만족하는 q 와 r 이 존재한다.
 - q 를 몫(quotient)라 하고, r 을 나머지(remainder)라고 한다.

공약수와 최대공약수

- a 와 b 의 공통된 약수 d 가 존재할 때, d 를 a 와 b 의 공약수(common divisor)라고 한다.
- 공약수의 성질은 아래와 같다.
 - $d \mid a$ 이고 $d \mid b$ 이면 $d \mid (ax + by)$ 이다.
 - $a \mid b$ 이고 $b \mid a$ 이면 $a = \pm b$ 이다.
- a 와 b 의 공약수 중 가장 큰 수를 a 와 b 의 최대 공약수라고 하고, $\gcd(a, b)$ 로 표현한다.
- 최대 공약수의 성질은 아래와 같다.
 - $\gcd(a, b) = \gcd(|a|, |b|)$
 - $\gcd(ka, a) = a$ (단, k 는 정수)

서로소와 소인수 분해

- 두 정수 a 와 b 에 대해 $\gcd(a, b) = 1$ 이면 a 와 b 를 서로소(relatively prime)라 하고, $a \perp b$ 로 나타낸다.
- 합성수 a 가 존재할 때, 소수 p_i 와 양의 정수 e_i 를 통해 다음의 형태로 a 를 표현할 수 있다.
 - $a = p_1^{e_1} p_2^{e_2} \cdots p_n^{e_n}$
 - 이러한 합성수 표현 방식을 소인수분해라고 한다.

모듈러 연산

- 모듈러 연산: 나머지를 구하는 연산
 - 즉, 나머지 정리 $a = qn + r$ 에서 음이 아닌 정수 r 을 구하는데 사용하는 연산이다.
 - C++에서는 모듈로 연산자 기호로 "%"를 사용한다.
 - `int r = a % n; // r은 a를 n으로 나눈 나머지 값`
- 합동식: n 으로 나누었을 때 같은 나머지 값을 가짐을 표현한 식
 - 쓰기: $r \equiv a \pmod{n}$ (n 을 합동의 법(modulus)라고 함)
 - 읽기: " r 은 법 n 에 대해 a 와 합동이다."
 - 즉, $r \bmod n = a \bmod n$ 와 같은 뜻을 지님

모듈러 연산

- 모듈러 연산에는 다음과 같은 성질이 존재한다.

- $(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$

- $(a - b) \bmod n = ((a \bmod n) - (b \bmod n)) \bmod n$

- $(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n$

- 나눗셈의 경우도 존재하긴 한다.

- $(a \div b) \bmod n = (a \cdot (b^{n-2} \bmod n)) \bmod n$ (단, $b \perp n$ 이고 n 은 소수)

- 왜 인지 알고 싶다면 합동식의 성질과 함께 모듈러 역원(modular inverse)에 대해서 공부해보자.

※ 쉽게 설명하면 정수 a 와 곱이 법 m 에 대해 1과 합동이 되도록 하는 수 ($ax \equiv 1 \pmod{m}$)

모듈러 연산

- PS에서 모듈러 연산의 활용

- 오버플로우 방지용으로 "구하려는 수를 M으로 나눈 값을 출력하시오." 라는 문구가 있는 문제

※ 주로 DP나 분할정복에서 사용됨. (뒤에 배울 이항계수나 빠른 거듭제곱에도 적용 가능)

- 순환이나 주기가 있는 문제 ($a[n-1]$ 과 $a[0]$ 이 연결되어 있는 문제, 시계, 요일 등)
- 해싱 기법을 사용하는 알고리즘에도 적용 가능 (대표적으로 라빈-카프 알고리즘)

- 법으로 자주 사용되는 수

- 998,244,353 (특징: 소수, 원시근이 3인 대표적인 수)
- 1,000,000,007 (특징: 소수)

```
const int m = 1e9 + 7;
int dp[1010101];

int fibo(int n) { // DP를 사용하여 n번째 피보나치 수열을 구하는 함수
    if (n <= 1) return 1;
    if (dp[n] != -1) return dp[n];
    return ((fibo(n - 1) % m) + (fibo(n - 2) % m)) % m;
}
```

Square Root 테크닉

- 핵심 아이디어: 자연수 n 에 대해 2 이상 $\sqrt{n}$ 이하 모든 자연수로 나누어지지 않는다면 n 은 소수
 - 소수가 아니라면 $n = ab$ 인 $1 < a, b < n$ 인 자연수 a, b 가 존재한다는 뜻인데, 둘 다 $\sqrt{n}$ 보다 크다면 $ab > n$ 으로 모순됨.
- 해당 아이디어로 아래 3가지 문제를 $O(\sqrt{n})$ 의 시간 복잡도로 간단히 해결 가능하다.
 - 소수 판별
 - 약수 찾기
 - 소인수분해

Square Root 테크닉

- 소수 판별

- n 이 주어졌을 때, n 이 $1 < x \leq \sqrt{n}$ 를 만족하는 자연수 x 에 대해 나누어 떨어지지 않음을 보이면 n 은 소수

```
bool isPrime(int n) {  
    for (int x = 2; x * x <= n; x++) {  
        if (n % x == 0) return false;  
    }  
    return true;  
}
```

Square Root 테크닉

- 약수 찾기

- $1 < x \leq \sqrt{n}$ 를 만족하는 n 의 약수 x 에 대해 n/x 또한 n 의 약수이다.
- $1 < x \leq \sqrt{n}$ 이고, $\sqrt{n} \leq n/x \leq n$ 이므로, $\sqrt{n}$ 이하의 모든 약수(즉, 모든 x)를 찾으면 n 의 모든 약수를 구할 수 있다.

```
vector<int> getDivisor(int n) {  
    vector<int> divisors;  
    for (int x = 2; x * x <= n; x++) {  
        if (n % x == 0) {  
            divisors.push_back(x);  
            if (x * x < n) divisors.push_back(n / x);  
        }  
    }  
    return divisors;  
}
```

Square Root 테크닉

- 소인수분해

- $1 < x \leq \sqrt{n}$ 인 모든 자연수 x 에 대해 작은 수 부터 다음과 같이 순회해준다.

[n 이 x 로 나누어 떨어지면 x 는 n 의 소인수라는 뜻이므로, n 을 x 로 나누어 떨어지지 않을 때 까지 나누어 준다.]

- 순회가 끝나고 n 이 1이라면 소인수 분해가 끝난 것이고, 1보다 크다면 n 또한 소수이므로 이를 소인수에 포함시켜준다.

```
vector <pair <int, int>> primeFactorization(int n) {  
    int N = n;  
    vector <pair <int, int>> result;  
    for (int x = 2; x * x <= n; x++) {  
        if (N % x == 0) {  
            int count = 0;  
            while (N % x == 0) N /= x, count++;  
            result.emplace_back(x, count);  
        }  
    }  
    if (N > 1) result.emplace_back(N, 1);  
}
```

에라토스테네스의 체

- n 이하의 소수를 $O(n \log \log n)$ 의 복잡도로 구할 수 있는 간단한 알고리즘
 - $\sqrt{n}$ 이하의 소수들을 작은 것 부터 순서대로 보면서 배수들을 지워가는 방식

에라토스테네스의 체

- 시뮬레이션: $n = 100$ 일 때

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

에라토스테네스의 체

- 우선 소수도, 합성수도 아닌 수 1을 지운다.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

에라토스테네스의 체

- 제일 작은 소수 2부터 살펴본다.
 - 2로 구성된 합성수 즉, 2의 배수를 제거한다.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

에라토스테네스의 체

- 2 다음으로 작은 소수 3을 살펴본다.
 - 마찬가지로 3의 배수를 제거해준다.

	2	3		5		7		9	
11		13		15		17		19	
21		23		25		27		29	
31		33		35		37		39	
41		43		45		47		49	
51		53		55		57		59	
61		63		65		67		69	
71		73		75		77		79	
81		83		85		87		89	
91		93		95		97		99	

에라토스테네스의 체

- 다음으로 작은 소수 5을 살펴본다.
 - 5의 배수를 제거해주자.

	2	3		5		7			
11		13				17		19	
		23		25				29	
31				35		37			
41		43				47		49	
		53		55				59	
61				65		67			
71		73				77		79	
		83		85				89	
91				95		97			

에라토스테네스의 체

- 마지막으로 소수 7을 살펴본다.
 - 7의 배수를 제거해준다.
 - 이 때 11은 소수이지만, $\sqrt{100} = 10$ 보다 크므로 7까지만 살펴본다

	2	3		5		7			
11		13				17		19	
		23						29	
31						37			
41		43				47		49	
		53						59	
61						67			
71		73				77		79	
		83						89	
91						97			

에라토스테네스의 체

- $\sqrt{100}$ 이하의 소수들의 배수를 제거한 결과
 - 따라서 해당 표에 남아있는 수들이 100 이하의 소수이다.

	2	3		5		7			
11		13				17		19	
		23						29	
31						37			
41		43				47			
		53						59	
61						67			
71		73						79	
		83						89	
						97			

에라토스테네스의 체

- 이를 그대로 구현하게 되면 다음과 같다.
 - 9번째 줄: 이 때 j 를 $i \cdot i$ 로 초기화하는 이유는 $i \cdot i$ 미만의 배수들은 이미 i 이전의 소수들에 의해 제거되었기 때문이다.

```
vector<int> eratosSieve(int n) {  
    vector<int> primeNumber;  
    vector<bool> isPrime(n + 1, true);  
  
    isPrime[0] = isPrime[1] = false;  
    for (int i = 2; i * i <= n; i++) {  
        if (isPrime[i] == true) {  
            primeNumber.push_back(i);  
            for (int j = i * i; j <= n; j *= i) {  
                isPrime[j] = false;  
            }  
        }  
    }  
    return primeNumber;  
}
```

유클리드 호제법

- 자연수 a, b 가 주어졌을 때 $\gcd(a, b)$ 를 구하는 알고리즘
- 핵심은 $a = bq + r$ 이라 하였을 때, $\gcd(a, b) = \gcd(b, r)$ 이다. ($a \geq b$)
 - $g = \gcd(a, b)$ 라고 하자. (즉, $g|a, g|b$) 이 때 공약수의 성질에 의해 $g | a - bq$ 가 성립하고, $r = a - bq$ 이므로 $g | r$ 이 성립한다.
 - 반대로 $g | b, g | r$ 이라면 $a = bq + r$ 이므로 공약수의 성질에 의해 $g | a$ 가 성립한다.
 - 이 때 $r = 0$ 이면 $b | a$ 라는 뜻이므로, b 가 a 의 최대공약수가 된다.
- 유클리드 호제법은 $O(\log a)$ 의 시간 복잡도를 가진다.
 - 이 때 $a \geq b$ 임을 가정하면 $q \geq 1$ 이고, $2r \leq (q + 1)r = qr + r \leq qb + r = a$ 이므로 $2r \leq a$ 이다.
 - 그러므로 $br \leq ab(1/2)$ 이고, 반복할수록 두 인자의 곱이 절반씩 줄어든다는 것을 알 수 있다.
 - 따라서 유클리드 호제법은 로그 시간에 최대 공약수를 구할 수 있다.

유클리드 호제법

- 구현
 - 반복문 또는 재귀함수로 쉽게 구현 가능

```
int gcd(int a, int b) {  
    if (b == 0) return a;  
    return gcd(b, a % b);  
}
```

```
int gcd(int a, int b) {  
    while (b > 0) {  
        int r = a % b;  
        a = b;  
        b = r;  
    }  
    return a;  
}
```

유클리드 호제법

- STL
 - C++ 11 버전에서는 GNU 컴파일러(g++)에서 자체 구현된 `std::__gcd()` 함수 제공
 - C++17 버전에서는 `std::gcd()`라는 표준 함수로 제공

```
1  #include <iostream>
2  #include <algorithm>
3  #include <numeric>
4  using namespace std;
5
6  int main() {
7      int a = 382824, b = 651222234;
8
9      cout << std::gcd(a, b) << endl;    // C++ 17 <numeric>
10     cout << std::__gcd(a, b) << endl; // GNU 컴파일러 C++ 11 <algorithm>
11
12     return 0;
13 }
```

이항 계수

- 이항식을 이항 정리로 전개했을 때 각 항의 계수. $\binom{n}{k}$ 로 표기.

- 이항식: $(x + a)^n$ 형태의 식

- 이항정리: $(x + a)^n = \sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$

- 이항계수 : $x^r a^{n-r}$ 의 계수 (즉, $\binom{n}{k}$)

- 이항계수 $\binom{n}{k} = \frac{n!}{r!(n-r)!}$ 이다. (증명 생략)

- 즉, 조합의 정의와 같음. (n 개중에서 k 개를 뽑는 경우의 수)

이항 계수

- 파스칼의 삼각형
 - 이항식을 거듭제곱한 것을 삼각형 형태로 나타낸 것을 뜻함.

Exponent

Pascal's Triangle

Binomial Expansion

0

1

$$(a+b)^0 = 1$$

1

1 1

$$(a+b)^1 = 1a + 1b$$

2

1 2 1

$$(a+b)^2 = 1a^2 + 2ab + 1b^2$$

3

1 3 3 1

$$(a+b)^3 = 1a^3 + 3a^2b + 3ab^2 + 1b^3$$

4

1 4 6 4 1

$$(a+b)^4 = 1a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + 1b^4$$

5

1 5 10 10 5 1

$$(a+b)^5 = 1a^5 + 5a^4b + 10a^3b^2 + 10a^2b^3 + 5ab^4 + 1b^5$$

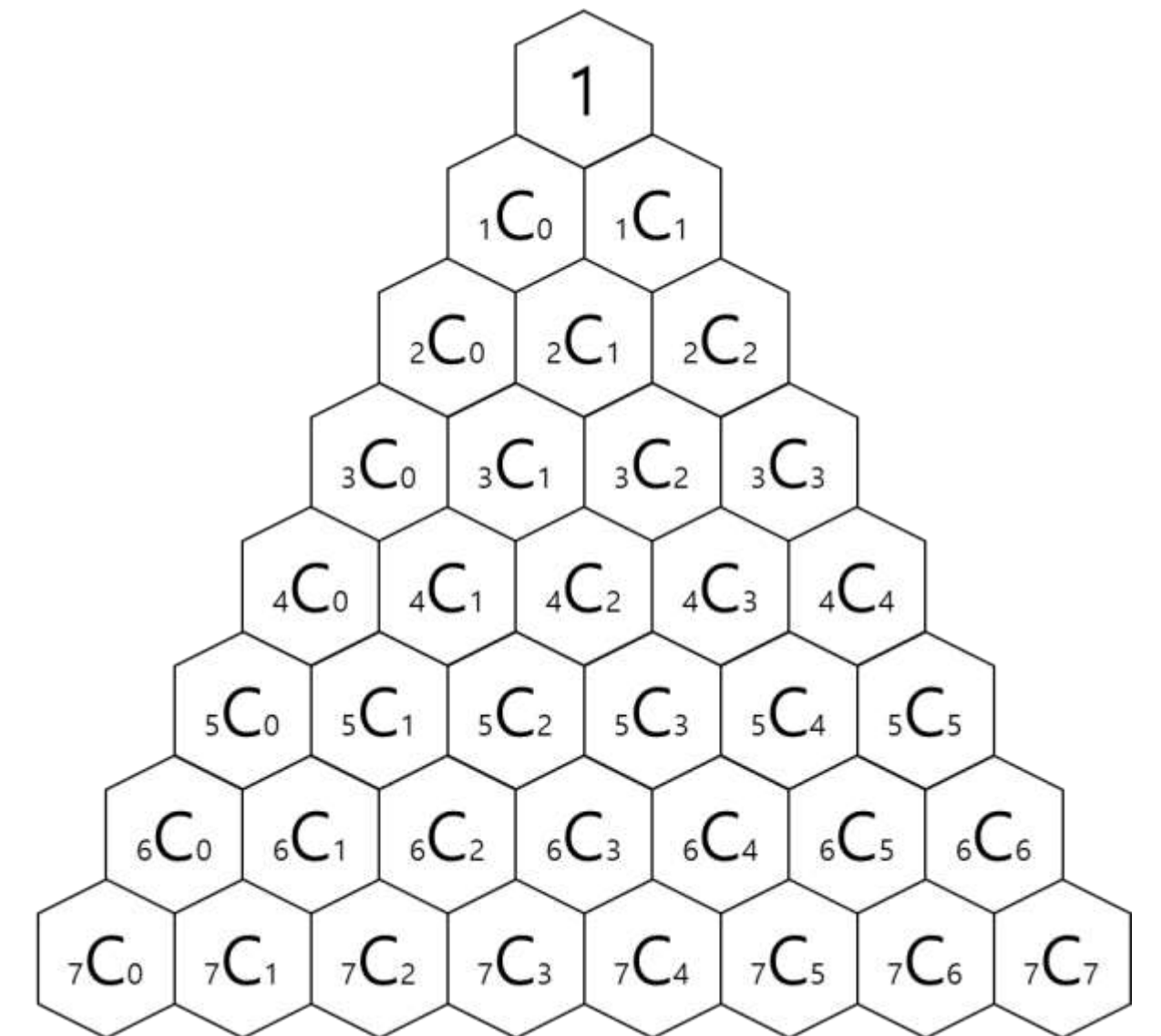
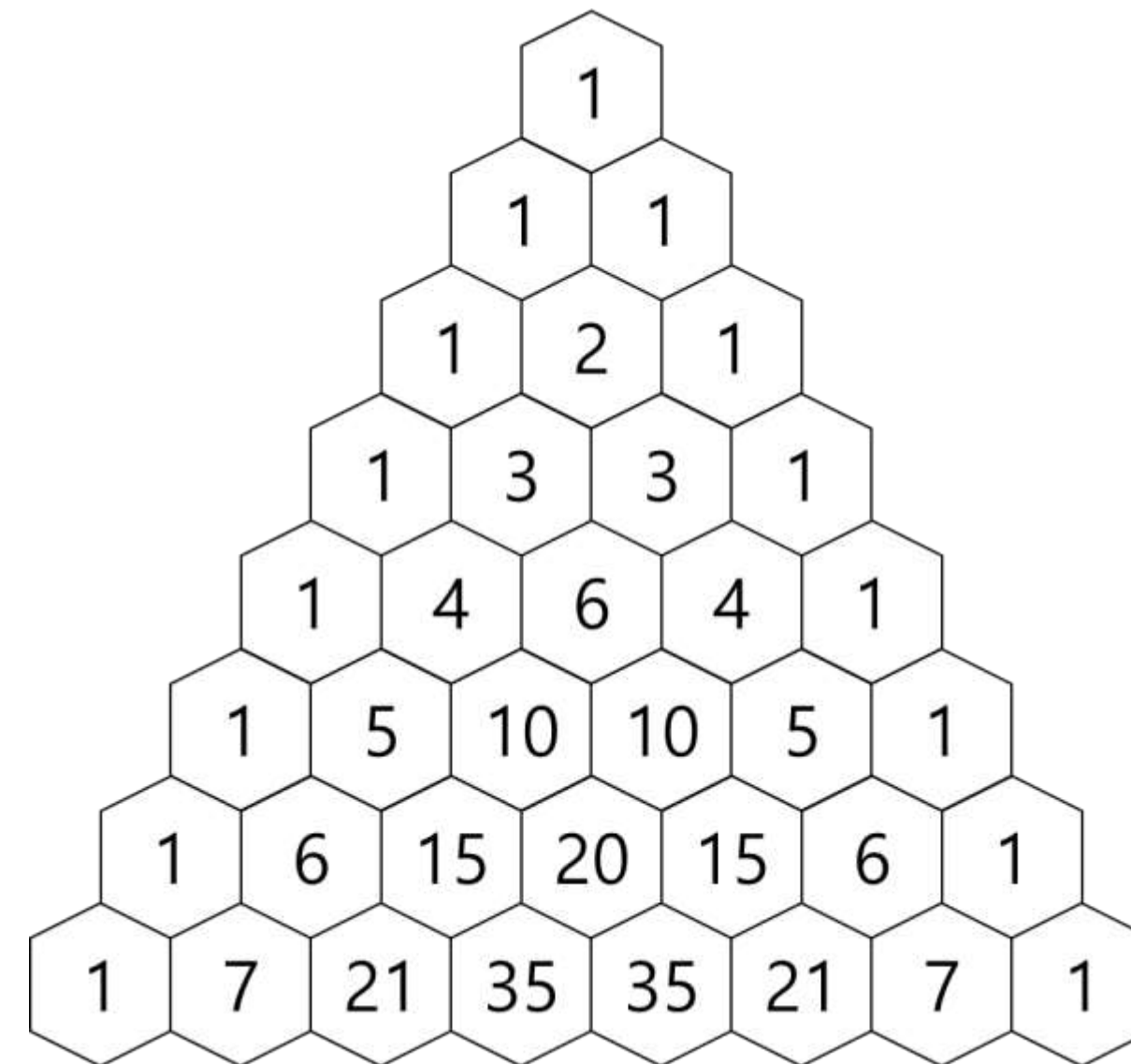
6

이항 계수

- 파스칼의 삼각형을 자세히 관찰하다 보면 다음과 같은 규칙을 발견할 수 있음.
- $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$
- 이 식을 DP로 활용하여 모든 이항계수를 $O(n^2)$ 에 구할 수 있음.

```
int dp[1010][1010];

int bino(int n, int k) {
    if (n < k) return 0;
    if (dp[n][k] == -1) {
        if (k <= 1) {
            dp[n][k] = 1;
        } else {
            dp[n][k] = bino(n - 1, k - 1) + bino(n - 1, k);
        }
    }
    return dp[n][k];
}
```



빠른 거듭제곱 (with 분할정복)

- a^n 는 a 를 n 번 제공한 것
 - 반복문을 활용하여 a 를 n 번 곱하면 $O(n)$ 에 이를 구할 수 있음.
- 분할정복을 통해 $O(\log n)$ 으로 줄일 수 있음.
 - n 이 짝수일 경우 $a^n = a^{n/2} \cdot a^{n/2}$
 - n 이 홀수일 경우 $a^n = a^{n-1} \cdot a^1$
 - 즉, n 이 짝수이면 $a^{n/2}$, n 이 홀수이면 a^{n-1} 을 구하면 a^n 을 구할 수 있음.
 - n 이 짝수일 때 절반으로 줄여가므로 a^n 을 $O(\log n)$ 에 구할 수 있다.

```
int pow(int a, int n) {  
    if (n == 0) return 1;  
    if (n % 2 == 0) {  
        int res = pow(a, n / 2);  
        return res * res;  
    } else {  
        int res = pow(a, n - 1);  
        return res * a;  
    }  
}
```

더 배우고 싶다면?

- 정수론

- 모듈로 역원, 확장 유클리드, 중국인의 나머지 정리, 밀러-라빈, 폴라드-로, 이산로그, 뫼비우스 함수 등..

- 조합론

- 순열, 중복 순열, 중복 조합, 포함 배제 원리, 카탈란 수, 벨 수, 기댓값 선형성, 생성함수 등..

- 기하

- 벡터공간, 선분교차, 볼록껍질, 회전하는 캘리퍼스, 반평면 교집합 등..

- 기타

- 게임이론 (스프라그 그룬디 등), 빠른 푸리에 변환(FFT)과 그 변형(NTT, FWHT) 등..